PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective 3
3. Product Purpose 5
4. System Functionalities 6
5. User Stories & Use Cases 8
6. Features Included (Scope Definition) 11
8. Assumptions & Constraints 15
9. Risks & Questions Throughout Development 19

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: Loom

Version: 1.0

1. Project Overview

- **Problem Statement:** Many early-stage founders, entrepreneurs, and small business owners have strong ideas but struggle to turn them into a working business system. In practice, they must handle business setup, customer communication, order tracking, marketing, and performance decisions separately. This creates heavy manual work, slows execution, and makes it difficult for non-technical users to run a business efficiently. LOOM addresses this by reducing the friction between business idea and business operation.
- **Target Domain:** AI-powered business automation and autonomous business management for small and medium businesses. The system is designed for businesses that need help with setup, customer support, sales handling, marketing execution, and operational decision-making
- **Proposed Solution Summary:** LOOM is an autonomous AI business engine that helps users create, run, and optimize a business through a simple prompt-based workflow. Instead of asking users to manually configure every part of the business, LOOM gathers the necessary business context through guided onboarding, then generates a working business setup, customer interaction flow, and marketing support layer.

2. Background & Business Objective

- **Background of the Problem:** Running a business usually requires many separate workflows: defining the offer, setting up customer communication, handling orders, creating marketing content, and making daily operational decisions. For early-stage founders and small business owners, these tasks are often done manually and with limited

© UMHackathon 2026
Email: umhackathon@um.edu.my

support. As a result, business setup takes too long, customer response becomes inconsistent, and growth decisions are made without enough structure or data.

This problem is especially strong for entrepreneurs who have ideas but do not have a full team, technical background, or time to manage everything at once. In many cases, they need multiple tools and repeated manual work just to operate a simple business. LOOM is designed to reduce that burden by combining setup, automation, and intelligence in one system.

- **Importance of Solving This Issue:** Solving this problem helps users move faster from idea to execution. Instead of spending days or weeks building a business workflow manually, users can begin operating with guided AI support in much less time. This is important because faster setup means faster testing, faster customer interaction, and faster learning.

  It also helps reduce the barrier for non-technical users. A founder should not need deep technical knowledge to create a functioning business system. By automating major parts of the process, LOOM allows users to focus on the business idea while the system handles much of the operational work.

- **Strategic Fit / Impact:** LOOM fits well with the direction of AI-native business tools, where intelligent systems are not just assistants but active operating layers for real business activity. The product combines conversational setup, operational management, and AI-based decision support in a way that is practical for small businesses.

  Its impact comes from reducing the gap between idea and execution. Instead of requiring separate tools for onboarding, order handling, marketing, and strategic planning, LOOM brings these functions together in a unified system. This makes it valuable for businesses such as food, physical products, digital products, and services, where speed, consistency, and automation matter.

© UMHackathon 2026

3. Product Purpose

3.1. Main Goal of the System: The main goal of LOOM is to provide an AI autonomous business operating system that helps users create, manage, and improve a business with minimal manual effort. The system is designed to take a simple business idea or prompt and turn it into a structured operational workflow that can handle setup, customer interaction, order processing, marketing support, and strategic decision-making. Rather than functioning as a basic chatbot, LOOM acts as an intelligent business layer that supports real business execution. It aims to reduce repetitive work, improve response speed, and help business owners make better decisions through automation and AI-based reasoning.

3.2. Intended Users (Target Audience):

- Product founders
- Entrepreneurs
- Small business owners
- Business/ services operators
- Low Code Builders/ no code expert

© UMHackathon 2026

4. System Functionalities

4.1. Description:

LOOM operates as an AI-driven business management system that supports users throughout the business lifecycle, from setup to daily operations. The system uses conversational onboarding to collect business information, create a business profile, and then connects that profile to operational features such as customer communication, store management, orders, marketing, payments, and analytics.

At a functional level, LOOM is designed to reduce manual work by combining business setup, AI assistance, and operational tracking in one connected workflow. It is not limited to answering questions; it helps execute core business tasks and generates decisions or outputs based on the business context stored in the system.

4.2. Key Functionalities: LOOM includes the following core functionalities:

- Business Onboarding: The system guides the user through a step-by-step conversational setup to collect the business name, category, operating mode, and business description.
- Customer Interaction AI: LOOM responds to customer messages automatically, supports natural conversation, and uses escalation logic when a message requires human attention.
- Order and Store Management: The system supports product listings, order tracking, order status monitoring, and a public store page where customers can browse and place orders.
- Marketing Insight Generation: LOOM can generate business insights, promotional copy, campaign ideas, and social media content based on business data and inventory context.
- Multi-Agent Decision Support: For strategic planning, LOOM simulates a board-style reasoning process using multiple AI roles, including CEO, CFO, and CMO, before producing a final recommendation.
- Telegram Integration: The system supports Telegram setup and webhook configuration so businesses can extend customer interaction beyond the web interface.

© UMHackathon 2026

- Business Dashboard: Owners can view summary metrics such as total orders, revenue, pending orders, and listed items from a central dashboard.
- Payments and Promotions Support: The database and business flow support payment methods, promo codes, and discount tracking to enable more complete commerce handling.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: LOOM uses Z AI GLM through the ILMU API as its main language model. This model is suitable because the product depends on structured prompting, business conversation, strategic reasoning, and content generation across multiple workflows. It is used in onboarding, customer chat, marketing generation, and multi-agent planning.

4.3.2. **Prompting Strategy:** The system uses a multi-step agentic prompting strategy rather than a single generic prompt. Different prompts are designed for different workflows:

- onboarding prompts for collecting business setup data
- customer support prompts for handling messages and escalation
- marketing prompts for generating posts, insights, and campaigns
- agent prompts for CEO, CFO, and CMO reasoning

This strategy fits LOOM because the system needs to behave differently depending on the task, while still staying consistent with the business context.

4.3.3. **Context & Input Handling**: LOOM handles input by combining the user request with relevant business context stored in the database, such as business name, category, current products, order data, and chat history. In the current codebase, context management is handled mainly through prompt construction and limited query selection rather than advanced chunking pipelines.

If input becomes too large, the practical behavior is that the system may rely on the most relevant recent context and can be limited by model token constraints. A future enhancement could introduce stronger summarization or chunking for long histories and larger business datasets.

© UMHackathon 2026
Email: umhackathon@um.edu.my

4.3.4. **Fallback & Failure Behavior:** LOOM includes basic fallback handling when the model fails or returns unusable output. For example:

- onboarding returns an error message if the AI call fails
- Customer Chat returns a safe apology message if generation fails
- marketing generation returns a structured error response
- Multi-agent planning can emit error states if one of the agent calls fails

This gives the system a graceful failure path, though more advanced retry and human escalation controls could be added later for stronger reliability.

5. User Stories & Use Cases

- **User Stories:**
  - As a founder, I want to describe my business ideas in simple language so that LOOM can set up my business profile and operating mode quickly.
  - As a business owner, I want LOOM to handle customer chat automatically so that I can respond faster without being online all the time.
  - As a store operator, I want to manage products and orders in one dashboard so that daily operations are easier to track.
  - As a marketer, I want AI-generated campaign ideas and social posts so that I can launch promotions with less effort.
  - As a decision maker, I want multi-agent strategic recommendations so that growth plans consider revenue, risk, and customer acquisition together.
  - As an owner, I want escalation alerts for sensitive customer issues so that I can step in when needed.
  - As a customer, I want a smooth public store experience so that I can browse items and place orders easily.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Use Cases (Main Interactions):**

P 1: Prompt-to-Business Setup

Actor: Business owner
Flow:

1. User starts onboarding chat.
2. LOOM asks for business name, category, mode, and description.
3. User provides answers step by step.
4. LOOM validates and creates business profile and settings.
5. User is redirected to the business dashboard.

Use Case 2: Customer Chat and Escalation

Actor: Customer, LOOM AI, Owner

Flow:

1. Customer sends a message through web chat.
2. LOOM reads business context and recent session history.
3. LOOM generates a response automatically.
4. If message severity is medium or high, LOOM logs escalation status.
5. Owner can review and intervene when necessary.

Use Case 3: Product Browsing and Order Placement

Actor: Customer

Flow:

© UMHackathon 2026
Email: umhackathon@um.edu.my

1. Customer opens the public store page.
2. Customer views available products and pricing.
3. Customer places an order and provides required details.
4. Order is recorded and appears in the owner dashboard.
5. Owner updates order status through operations flow.

Use Case 4: Marketing Content and Strategy Generation

Actor: Owner/Marketing operator

Flow:

1. User requests insight, post, promo, or campaign strategy.
2. LOOM gathers business data such as products and order signals.
3. LOOM generates tailored marketing output.
4. Result is displayed and can be reused by the owner.
5. Generated insight is stored for history and reference.

Use Case 5: Multi-Agent Strategic Planning

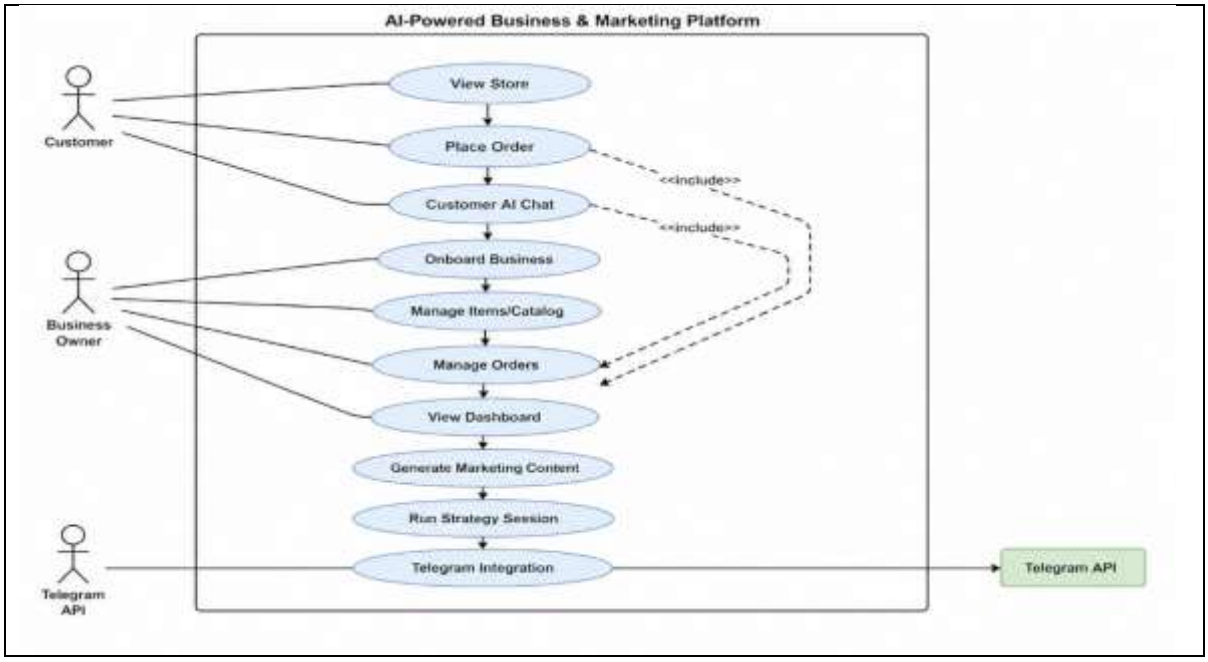
Actor: Owner, AI Agent Board

Flow:

1. User submits goal and budget.
2. CEO agent proposes growth direction.
3. CFO agent evaluates cost, risk, and financial impact.
4. CMO agent recommends channel and campaign approach.
5. LOOM synthesizes all perspectives into one action plan.

- **Use Case Diagram:**

© UMHackathon 2026



6. Features Included (Scope Definition)

The following features are included in LOOM's current product scope based on implemented system behavior and core platform intent.

1. AI-Guided Business Onboarding
    - Conversational onboarding flow to collect business name, category, operating mode, and description.
    - Automatic creation of business profile and default business settings after setup completion.
2. Business Profile and Mode Configuration
    - Support for key business categories: food, physical products, digital products, and services.
    - Support for operating modes such as landing-focused, Telegram-focused, or both.
3. Dashboard and Operational Visibility

© UMHackathon 2026
Email: umhackathon@um.edu.my

- Central dashboard showing key metrics such as orders, revenue, pending orders, and listed items.
- Recent order activity visibility for fast operational monitoring.

4. Product and Catalog Management
    - Add and manage business items with name, description, price, stock, and availability state.
    - Optional item-level discount metadata and promo labels supported in data model.

5. Public Storefront and Customer Ordering
    - Public business store page where customers can browse available items.
    - Customer order placement flow with persisted order and order-item records.
    - Support for receipt and payment-related order fields.
6. AI Customer Interaction and Escalation
    - AI-powered customer chat that uses business context and session history.
    - Escalation-level detection (low/medium/high) for sensitive or complaint-based interactions.
    - Escalation actions logged for owner awareness and follow-up.
7. Marketing Intelligence and Content Generation
    - AI-generated growth insights and short strategy outputs based on business data.
    - AI-generated social content and promotional campaign suggestions.
    - Marketing outputs stored as insights for later review.
8. Multi-Agent Strategic Decision Flow
    - Multi-agent board simulation for strategic planning with role-based perspectives (CEO, CFO, CMO).
    - Synthesis of role outputs into a final actionable plan tied to business goal and budget.
9. Telegram Integration and Messaging Setup
    - Telegram webhook setup flow for bot-based customer interaction channels.
    - Telegram session data model support for tracking conversation/cart context.

© UMHackathon 2026
Email: umhackathon@um.edu.my

10. Payment and Promotion Support
    - Payment methods management model (e.g., QR or bank transfer details).
    - Promo code model with discount rules, usage tracking, and validity constraints.
11. Data, Security, and Access Control Foundations
    - Supabase-backed persistence for businesses, items, orders, messages, insights, actions, and settings.
    - Row-level security policies to restrict business data access to authorized owners.
    - Authenticated/anonymous access patterns for private dashboard vs public store experiences.

7. Features Not Included (Scope Control)

To keep the product focused and technically reliable, the following features are currently out of scope for this version of LOOM:

1. Full Real-Time Ad Platform Execution
    - LOOM can generate campaign strategy and ad copy, but direct end-to-end execution on external ad platforms (for example automatic campaign publishing, bidding control, and budget pacing across all networks) is not fully implemented as a production-grade integration.
2. Fully Implemented 4-Agent Board (Including CTO)
    - The product vision provides CEO, CFO, CMO, and CTO.
    - Current implemented strategic flow clearly supports CEO, CFO, and CMO roles.
    - A dedicated CTO reasoning pipeline is not yet fully implemented in the same operational depth.
3. Autonomous Financial Transactions and Accounting Operations
    - LOOM supports payment method setup and order/payment-related fields.
    - It does not execute autonomous financial transactions, accounting reconciliation, tax filing, or banking operations.
4. Legal, Compliance, and Regulatory Automation

© UMHackathon 2026
Email: umhackathon@um.edu.my

- The system does not automatically perform legal review, compliance filing, regulatory reporting, or contract generation with legal guarantees.

5. Advanced External Market Data Pipelines
    - LOOM uses internal business data and AI reasoning for recommendations.
    - Continuous ingestion from broad external real-time market intelligence providers is not yet implemented as a core production feature.
6. Fully Automated Closed-Loop Self-Execution Across All Modules
    - The product supports analysis, recommendations, and selected automation flows.
    - A complete autonomous loop that independently executes every business function without user confirmation is not fully enabled by default.
7. Unified Multi-Business Control Center
    - The data model supports multiple businesses, but a full portfolio-level management interface with cross-business optimization and automatic resource reallocation is not yet fully implemented.
8. Enterprise Collaboration and Role Hierarchy
    - Advanced team collaboration features such as granular role-based approvals, multi-admin workflows, and enterprise permission matrix controls are not in this scope version.
9. Native Mobile Applications
    - The current product is web-based.
    - Dedicated iOS/Android native app implementations are not included in current scope.
10. Guaranteed Human-in-the-Loop Workflow Builder Everywhere
    - Some escalation and manual intervention paths exist.
    - A unified, explicit manual override orchestration panel for every autonomous action is not fully implemented as a single comprehensive module.

© UMHackathon 2026
Email: umhackathon@um.edu.my

8. Assumptions & Constraints

This chapter defines the operational assumptions and system limitations that shape LOOM's current implementation and deployment behavior.

1. **LLM Cost Constraint:** LOOM relies on repeated model inference across onboarding, customer chat, marketing generation, and multi-agent strategic reasoning. This creates variable token usage based on conversation depth and context length.

   Assumptions:

   Average usage is manageable when prompts are task-specific and context is selectively scoped.

   High-frequency marketing generation or long multi-turn customer sessions increase cost significantly.

   Cost-control decisions in the current design:

   - Prompts are separated by function (onboarding, chat, marketing, strategy) to avoid sending unnecessary context.
   - Recent history and business-relevant data are used instead of full unbounded histories.
   - Some routes cap response length behavior through max token configuration.
2. **Technical Constraints:** LOOM's core operation depends on a defined cloud stack and service availability.

   Current constraints:

   - Supabase is required for authentication, database persistence, and access control policies.
   - ILMU/GLM model API availability directly affects AI feature reliability.
   - Telegram-related capabilities depend on valid bot token and webhook setup.
   - Public store and private dashboard flows rely on correct RLS and table policy configuration.
   - Missing environment variables (for example API keys) will degrade AI flows.

© UMHackathon 2026
Email: umhackathon@um.edu.my

3. **Performance Constraints:** LOOM performance is influenced by AI latency, database query cost, and concurrent route usage.

   Observed/expected constraints:

   - Multi-agent reasoning requires multiple sequential/parallel model calls, increasing response time.
   - AI-heavy endpoints may experience higher latency during peak usage.
   - Streaming improves perceived responsiveness in strategy workflows but does not remove total generation time.
   - Large item catalogs or extensive order history can increase context assembly cost before generation.

4. **User Input:** LOOM accepts unstructured user prompts and conversational input but still depends on minimum context quality.

   Assumptions:

   - The model performs best when user intent is clear and business details are sufficiently specified.
   - Very vague prompts reduce output precision.
   - Very long input/history may require context trimming at application level.
   - Current handling behavior:
   - Context is assembled from business profile, items, recent messages, and operational data.
   - Input is not yet governed by a fully formalized chunking/summarization pipeline across all endpoints.
   - Oversized requests can result in slower responses, truncation effects, or model-side degradation.

© UMHackathon 2026

5. **Reliability and Failure-Mode Constraints**

   The system includes basic graceful fallback behavior but not full fault orchestration across all modules.

   Constraints:

   - If model generation fails, routes return safe fallback/error responses.
   - Retry strategy is limited and route-specific.
   - Human escalation exists in selected customer chat scenarios, but a unified intervention dashboard is not fully implemented.

6. **Automation Scope Constraints**

   LOOM is designed as an autonomous engine with explainability, but current automation is partially supervised.

   Assumptions:

   - Users still provide initial prompts, business setup details, and selected operational inputs.
   - Strategy outputs may require owner approval before real-world execution.
   - Some outputs are simulated intelligence artifacts rather than direct third-party execution results.

7. **Security and Access Constraints**

   LOOM uses role-based and row-level access assumptions for business data safety.

   Constraints:

   - Correct RLS policies are mandatory to prevent cross-tenant access issues.
   - Public store access intentionally exposes only necessary storefront data.
   - Misconfiguration of policies or storage rules can introduce security risk.

© UMHackathon 2026

8. Deployment and Environment Constraints

   System behavior depends on consistent environment setup across development and production.

   Constraints:

   - Required secrets and service keys must be configured correctly.
   - Webhook URLs and app domain settings must be valid for Telegram flows.
   - Database schema and migrations must remain synchronized with route expectations.

© UMHackathon 2026

Email: umhackathon@um.edu.my

9. Risks & Questions Throughout Development

This chapter captures key product and engineering risks that may affect LOOM's reliability, scalability, and business impact, together with the controls used to reduce those risks.

1. Decision Similarity
   - How are we ensuring that LOOM does not generate overly similar business strategies, campaign ideas, or recommendations across different users and business categories?

How we address it:

- LOOM injects business-specific context into prompts, including business category, product set, order patterns, revenue signals, user goal, and budget. Outputs are generated from context-aware prompts instead of generic templates, and role-based reasoning (CEO/CFO/CMO) creates perspective diversity before final synthesis.

Remaining improvement:

- Introduce output diversity scoring and recommendation history checks to further reduce repeated strategy patterns over time.

2. Execution Stability

© UMHackathon 2026
Email: umhackathon@um.edu.my

- If LOOM generates an action plan that is partially impractical in real operations (budget, staffing, channel limitations), does the user need to manually correct it before execution, and how much correction is expected?

How we address it:

- LOOM uses feasibility-aware prompts with explicit constraints (for example budget and available business context), and the system treats strategic outputs as guided action plans rather than forced autonomous execution. Critical decisions can remain user-controlled through review and confirmation flows.

Remaining improvement:

- Add explicit feasibility validation layers (cost/effort/risk checks) and pre-execution guardrails for high-impact actions.

3. Input Interpretation
   - What happens if LOOM cannot properly contextualize unclear, incomplete, or conflicting business prompts during onboarding or strategic planning, and how does it recover safely?

How we address it:

- LOOM uses step-based onboarding to collect required fields in sequence, and fallback error behavior is implemented for model failure cases. In customer chat, escalation detection and safe-response handling reduce risk when confidence is low or messages are sensitive.

Remaining improvement:

- Add formal confidence thresholds, clarification loops for ambiguous prompts, and structured retries before final output commitment.

© UMHackathon 2026